COMP 484/584 Homework 2

Your Name:
NetID:

2026

In this homework, there are two problems that must be solved by hand and one coding implementation problem.

Submission. You should submit:

- A PDF report containing your answers and analysis. For Problems 1–2, you may choose either handwritten work or LaTeX.
- The completed Jupyter notebook with your implementation `HW1.ipynb`.
- Please submit your homework (a PDF + a .ipynb) as **one single `.zip` file** named with your NetID:

```
NLP_HW1_<NETID>.zip
```

Please make sure that your report is **exported as a PDF**. Do **not** submit Word documents or LaTeX source files.

You must submit the `.zip` file to Canvas before the deadline.

Problem 1 (25 pts): RNN Language Model

Let the vocabulary be $\mathcal{V}$ with size $|\mathcal{V}| = V$. We model a sentence with special tokens:

$$x_0 = \langle s \rangle, \quad x_1, x_2, \ldots, x_T, \quad x_{T+1} = \langle e \rangle,$$

where each $x_t \in \mathbb{R}^V$ is a **one-hot** vector.

1

Model Definition

The learnable parameters of the model are $\theta = \{E, W_h, W_e, b_1, U, b_2\}$, defined as:

- $E \in \mathbb{R}^{d \times V}$: Word embedding matrix.

- W_h, W_e, b_1: Parameters for the recurrent update step.
- U, b_2: Parameters for the output projection distribution.

We use the following "shifted" RNN-LM convention: at time t we **input the previous token** x_{t-1} and **predict the current token** x_t. Initialize $h_0 = \mathbf{0}$. For $t = 1, 2, \ldots, T+1$:

$$\begin{aligned} e_{t-1} &= Ex_{t-1} \\ a_t &= W_h h_{t-1} + W_e e_{t-1} + b_1 \\ h_t &= \tanh(a_t) \\ z_t &= Uh_t + b_2 \\ \hat{y}_t &= \mathrm{softmax}(z_t) \in \mathbb{R}^V. \end{aligned}$$

Thus $\hat{y}_t$ represents the distribution over the next token:

$$\hat{y}_{t,j} \;=\; P(x_t = w_j \mid x_{0:t-1}).$$

Training Objective

Let the target be $y_t = x_t$ (one-hot). The loss at time t is the cross-entropy / negative log-likelihood:

$$L_t \;=\; -\sum_{j=1}^{V} y_{t,j} \log \hat{y}_{t,j}.$$

The total sequence loss is:

$$J \;=\; \sum_{t=1}^{T+1} L_t.$$

(a) (10 pts) Perplexity

Perplexity is defined (as in lecture slides) by:

$$\mathrm{PPL} \;=\; \exp\left(-\frac{1}{T+1} \sum_{t=1}^{T+1} \log P(x_t \mid x_{0:t-1}) \right).$$

(a) Prove that for the above model,

$$\mathrm{PPL} \;=\; \exp\left(\frac{J}{T+1}\right).$$

(b) Suppose the model predicts the uniform distribution at every step: $\hat{y}_{t,j} = \frac{1}{V}$ for all j and all t. Derive the perplexity in terms of V.

(b) (15 pts) Backpropagation Through Time : derive $\partial J / \partial W_h$

Define

$$\delta_t \;=\; \frac{\partial J}{\partial a_t} \in \mathbb{R}^H.$$

You may use (without proof) the standard softmax-cross-entropy identity:

$$\frac{\partial L_t}{\partial z_t} \;=\; \hat{y}_t - y_t.$$

Also recall the tanh derivative:

$$\frac{\partial}{\partial a} \tanh(a) = 1 - \tanh^2(a).$$

(a) Derive a recurrence for δ_t from $t = T + 1$ down to $t = 1$. Your final expression should be written using only W_h, U, $\hat{y}_t$, y_t, h_t, and δ_{t+1}. (You may assume $\delta_{T+2} = \mathbf{0}$.)

(b) Using your result from (a), derive an explicit summation expression for $\dfrac{\partial J}{\partial W_h}$ in terms of $\{\delta_t\}_{t=1}^{T+1}$ and $\{h_t\}_{t=0}^{T}$.

Problem 2 (30 pts): Understanding and Differentiating the LSTM Cell

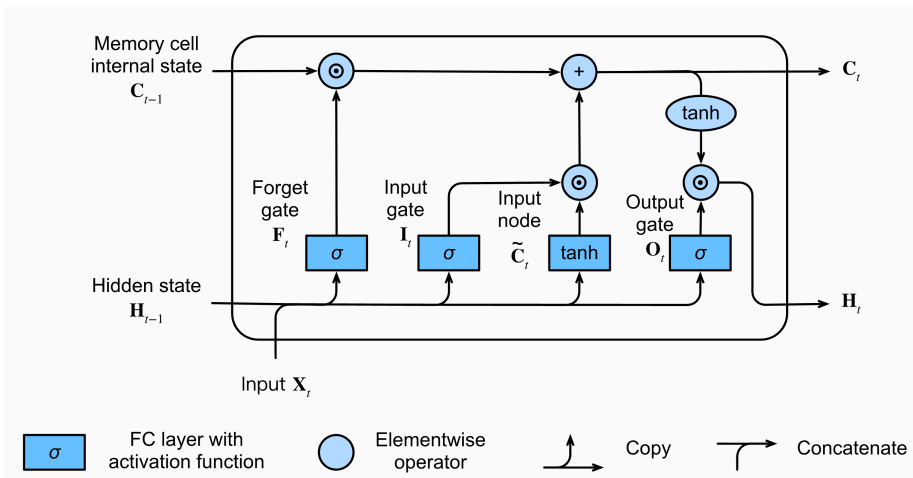

Figure 1: Computation graph of a single LSTM cell.

Consider the LSTM cell shown in Figure 1. You are not given the explicit equations. Instead, you must derive them directly from the computation graph.

Assume:

- Input $x_t \in \mathbb{R}^d$
- Previous hidden state $h_{t-1} \in \mathbb{R}^H$
- Previous cell state $c_{t-1} \in \mathbb{R}^H$
- Each gate is implemented as an affine transformation of the concatenation $[h_{t-1}; x_t]$

Let $J = \sum_{t=1}^{T} L_t$ be the total loss over a sequence.

(a) Forward Pass (10 pts)

Using only the computation graph:

1. Define appropriate mathematical notation for all intermediate variables.
2. Write the complete forward computation for time step t.
3. Clearly indicate:

- which operations are affine transformations,
- which operations are elementwise,
- which operations are nonlinearities.

4. Specify the dimensionality of each variable.

You must explicitly state any assumptions you make.

(b) Backward Pass: Cell-State Gradient Flow (10 pts)

Define:

$$\delta_t^h = \frac{\partial J}{\partial h_t}, \quad \delta_t^c = \frac{\partial J}{\partial c_t}.$$

1. Derive an expression for δ_t^c in terms of:

$$\delta_t^h, \quad \delta_{t+1}^c, \quad \text{and the relevant gate variables.}$$

2. Your final expression must clearly show the recursive dependence across time.
3. Identify which term is responsible for long-term gradient propagation.

(c) Gradient with Respect to Parameters (5 pts)

Choose the forget gate.

1. Derive $\frac{\partial J}{\partial W_f}$.
2. Express your answer as a summation over time.
3. Your final answer should be written in matrix form.

(d) Stability Analysis (5 pts)

Consider the case where the forget gate satisfies:

$$f_t \approx 1 \quad \text{for all } t.$$

1. Show what the Jacobian $\frac{\partial c_T}{\partial c_t}$ becomes.
2. Compare this with the corresponding Jacobian in a vanilla RNN.
3. Briefly explain why LSTMs mitigate the vanishing gradient problem.

Problem 3 (45 pts): Understanding and Implementing an LSTM from Scratch

In this problem, you will implement an LSTM cell directly from the computation graph shown in Figure 1. You may NOT use `torch.nn.LSTM` or `torch.nn.LSTMCell`.

You must implement all gates explicitly.

(a) Implement a Single LSTM Cell (5 pts)

Implement a class:

```
class LSTMCellScratch(nn.Module):
    def __init__(self, input_size, hidden_size):
        ...
    def forward(self, x_t, h_prev, c_prev):
        ...
```

Requirements:

- Use affine transformations of the concatenation $[h_{t-1}; x_t]$.
- Implement four gates: forget, input, output, candidate.
- Clearly separate affine operations and elementwise operations.
- Return h_t and c_t.

(b) Build a Sequence Model (10 pts)

Using your LSTM cell, implement an LSTM language model.

Your model should include:

- An embedding layer
- Your LSTM cell
- A linear output layer

The forward pass should return logits for all time steps.

(c) Train on a Synthetic Task (20 pts)

Train your model on a copy task:

Input sequence:

$$x_1, x_2, \ldots, x_T$$

Target sequence:

$$x_1, x_2, \ldots, x_T$$

Report:

- Training loss curve
- Final perplexity
- Gradient norm of the cell state over time

(d) (10 pts) Q&A

Describe the fundamental limitations of RNNs in NLP tasks, and explain how the Transformer architecture overcomes these issues. (minimum 100 words)